→ what is a Signal ?

→ Ignore, Default, userdefined

↓

Most of the Signals default action termination of procey to whom the Signal is delivered.

→ Standard Real time

1-32

Synchronous Signal

Asynchronous Signal.

SIGCHLD → child procey when it gets terminated

Parent procey → Releasing all the

resources assigned to the child process

SIGINT :-

ctrl+C →

SIGQUIT

ctrl+\

→ SIGINT Vs SIGQUIT

→ 9 & 19

SIGKILL & SIGSTOP

→ SIGSEGV → display the msg terminating the process.

↓

MMU

→ / → Divide by zero

SIGFPE → H/w

· Termination of a process

GDB

→ —

→ n

break point

keil Debugger

Debuggy

step into

breakpoint

SIGCONT

SIGSTOP

→ SIGNAL , SIGACTION

Registering the Signal action.

Q1 Signal Vs SIGACTION

flags

Mask

PINSEL → pin funct

Union {

int PINSEL0

Struct

{

2.

} 16 m

}

Q How to Send a Signal (cmd/systemcall) to the process

Kill (PID, Sig-num)

Alarm(5)

Alarm(10)

Spin_Lock :-

Advantage : data Safe.

→ Disadvantage :- CPU's Waiting time is increasing.

mutex (Mutual exclusion) :-

Char ch1, ch2;

mutex_init(&m1), mutex_init(&m2);

job1

mutex_lock(↑m1)

ch1 = —

} C.R./C.S.

mutex_unlock(↑m1)

job2

mutex_lock(↑m1)

ch1 = —

} C.S.

mutex_unlock(↑m1)

mutex_lock(↑m2)

ch2 = —

mutex_unlock(↑m2)

job3

mutex_lock(↑m2)

ch2 = —

mutex_unlock(↑m2)

mutex-init → This Library fn is to initialize the mutex object with default value (0)

↓

free state.
unlocked state.

→ mutex is a kernel object.

mutex_lock (&m1)

↳ checks m1 value

→ if m1 is 0 then it makes m1 as 1 (locked state) and Calling thread will cont....

→ else Calling thread will be blocked (put into waiting state)

mutex_unlock(&m1)

↳ m1 is made as 0